

ANEXO TÉCNICO

# DOCUMENTACIÓN TÉCNICA

---

Sistema de Gestión para Tienda de Videojuegos

---

**Proyecto:** Reto2CRUD\_DIN\_ADT

**Tecnología:** JavaFX · Hibernate · MySQL

**Versión:** 1.0 | **Fecha:** Mayo 2026

---

# ÍNDICE

---

## 1. Introduccion

## 2. Analisis

### 2.1. Definicion del sistema

#### 2.1.1. Determinacion del alcance del sistema: descripcion funcional

#### 2.1.2. Identificacion de usuarios

Usuario

Administrador

### 2.2. Especificacion de casos de uso (jerarquizada)

#### 2.2.1. Casos de uso de autenticacion y acceso

#### 2.2.2. Casos de uso de gestion de perfil

#### 2.2.3. Casos de uso de consulta de catalogo

#### 2.2.4. Casos de uso de compra

#### 2.2.5. Casos de uso de resenas

#### 2.2.6. Casos de uso de administracion del catalogo

#### 2.2.7. Casos de uso de administracion de usuarios

### 2.3. Modelo conceptual de datos (incluyendo cardinalidades)

Entidades principales

Cardinalidades

Representacion esquematica

### 2.4. Arquitectura de software de la solucion

#### 2.4.1. Capa de presentacion

#### 2.4.2. Capa de logica de aplicacion

#### 2.4.3. Capa de persistencia

#### 2.4.4. Base de datos

#### 2.4.5. Librerias y dependencias

#### 2.4.6. Flujo general de ejecucion

## 3. Inventario tecnico resumido

Paquetes

Pruebas

#### **4. Observaciones tecnicas**

#### **5. Conclusion**

# Entregable: Anexo de Documentacion Tecnica

---

## 1. Introduccion

---

Este documento describe tecnicamente el proyecto `Reto2CRUD_DIN_ADT`, una aplicacion de escritorio desarrollada en JavaFX para la gestion de una tienda de videojuegos. El sistema implementa autenticacion, gestion de usuarios, administracion de catalogo, carrito de compra, compras y resenas, apoyandose en Hibernate para el acceso persistente a datos sobre MySQL.

El objetivo del anexo es dejar definido el alcance funcional, los usuarios del sistema, los casos de uso principales, el modelo conceptual de datos y la arquitectura de software utilizada.

## 2. Analisis

---

### 2.1. Definicion del sistema

El sistema es una aplicacion cliente de escritorio con interfaz grafica JavaFX y almacenamiento persistente en base de datos relacional. La aplicacion se inicia en la ventana de login y, tras autenticarse, deriva el flujo segun el tipo de perfil identificado: usuario o administrador.

Evidencias tecnicas principales:

- Punto de entrada: `PRG/RetoFinal/src/main/Main.java`
- Controladores de interfaz: `PRG/RetoFinal/src/controller/`
- Entidades y persistencia: `PRG/RetoFinal/src/model/`
- Vistas FXML: `PRG/RetoFinal/src/view/`
- Configuracion Hibernate: `PRG/RetoFinal/src/hibernate.cfg.xml`

### 2.1.1. Determinacion del alcance del sistema: descripcion funcional

El alcance funcional actual del sistema cubre:

- Inicio de sesion para usuarios y administradores.
- Registro de nuevos usuarios.
- Creacion automatica de un administrador por defecto al arrancar la aplicacion.
- Consulta del catalogo de videojuegos.
- Filtrado de videojuegos por nombre, genero y plataforma.
- Gestion de carrito de compra en memoria durante la sesion.
- Registro de compras mediante pedidos persistidos en base de datos.
- Escritura de resenas de videojuegos por parte de usuarios.
- Modificacion del perfil del usuario autenticado.
- Eliminacion de la propia cuenta por parte del usuario.
- Gestion administrativa del catalogo: alta, modificacion y baja de videojuegos.
- Eliminacion administrativa de cuentas de usuario.
- Apertura de manuales y documentacion PDF desde la aplicacion.

Quedan fuera del alcance visible del proyecto:

- Aplicacion web o API REST.
- Pago real con pasarela externa.
- Cifrado de contraseñas.
- Generacion operativa de informes JasperReports, aunque las librerías están incluidas.

### 2.1.2. Identificacion de usuarios

#### Usuario

Perfil orientado al cliente final de la tienda. Puede:

- Registrarse.
- Iniciar sesion.
- Consultar el catalogo.
- Filtrar videojuegos.

- Anadir productos al carrito.
- Formalizar compras.
- Publicar resenas.
- Modificar sus datos.
- Eliminar su cuenta.

Entidad relacionada: `PRG/RetoFinal/src/model/User.java`

## Administrador

Perfil orientado a la gestion del sistema. Puede:

- Iniciar sesion.
- Consultar el catalogo.
- Filtrar videojuegos.
- Anadir videojuegos.
- Modificar videojuegos.
- Eliminar videojuegos.
- Eliminar cuentas de usuario.

Entidad relacionada: `PRG/RetoFinal/src/model/Admin.java`

## 2.2. Especificacion de casos de uso (jerarquizada)

### 2.2.1. Casos de uso de autenticacion y acceso

- CU-01. Iniciar sesion.
- CU-02. Registrarse como nuevo usuario.
- CU-03. Cerrar sesion y volver al menu principal.

Controladores relacionados:

- `LogInWindowController.java`
- `SignUpWindowController.java`
- `MenuWindowController.java`

### 2.2.2. Casos de uso de gestion de perfil

- CU-04. Modificar datos personales del usuario.
- CU-05. Eliminar cuenta propia.

Controladores relacionados:

- `ModifyWindowController.java`
- `DeleteAccountController.java`

### 2.2.3. Casos de uso de consulta de catalogo

- CU-06. Listar videojuegos disponibles.
- CU-07. Filtrar videojuegos por criterios.
- CU-08. Ver informacion detallada del videojuego seleccionado.

Controladores relacionados:

- `ShopWindowController.java`
- `AdminShopController.java`

### 2.2.4. Casos de uso de compra

- CU-09. Anadir videojuego al carrito.
- CU-10. Modificar cantidades del carrito.
- CU-11. Eliminar producto del carrito.
- CU-12. Confirmar compra.
- CU-13. Registrar pedido en base de datos.

Controlador relacionado:

- `CartController.java`

### 2.2.5. Casos de uso de resenas

- CU-14. Escribir resena de un videojuego.
- CU-15. Validar que no exista una resena duplicada para el mismo usuario y videojuego.

Controlador relacionado:

- `ReviewController.java`

### 2.2.6. Casos de uso de administracion del catalogo

- CU-16. Dar de alta un videojuego.
- CU-17. Modificar un videojuego existente.
- CU-18. Eliminar un videojuego.

Controladores relacionados:

- `AddGamesAdminController.java`
- `ModifyGameAdminController.java`
- `AdminShopController.java`

### 2.2.7. Casos de uso de administracion de usuarios

- CU-19. Buscar usuarios para administracion.
- CU-20. Eliminar una cuenta de usuario desde el perfil administrador.

Controlador relacionado:

- `DeleteAccountAdminController.java`

## 2.3. Modelo conceptual de datos (incluyendo cardinalidades)

El modelo del sistema se organiza alrededor de perfiles, videojuegos, pedidos y resenas.

### Entidades principales

- `Profile` : entidad abstracta base con `userCode` , `username` , `password` , `email` , `name` , `telephone` , `surname` .
- `User` : hereda de `Profile` y anade `gender` y `cardNumber` .
- `Admin` : hereda de `Profile` y anade `currentAccount` .
- `Videogame` : representa cada producto del catalogo.
- `Order` : representa una compra de un usuario sobre un videojuego.
- `Review` : representa una valoracion de un usuario sobre un videojuego.

### Cardinalidades

- `Profile` 1..1 -> `User` o `Admin` por herencia `JOINED` .
- `User` 1 -> N `Order` .



- **User** 1 -> N **Review** .
- **Videogame** 1 -> N **Order** .
- **Videogame** 1 -> N **Review** .
- **Order** N -> 1 **User** .
- **Order** N -> 1 **Videogame** .
- **Review** N -> 1 **User** .
- **Review** N -> 1 **Videogame** .

## Representacion esquematica

```
classDiagram
    Profile <|-- User
    Profile <|-- Admin

    User "1" --> "0..*" Order
    Videogame "1" --> "0..*" Order
    User "1" --> "0..*" Review
    Videogame "1" --> "0..*" Review

    class Profile {
        +int userCode
        +String username
        +String password
        +String email
        +String name
        +String telephone
        +String surname
    }

    class User {
        +String gender
        +String cardNumber
    }

    class Admin {
        +String currentAccount
    }

    class Videogame {
```

```
+int idVideogame
+String companyName
+GameGenre gameGenre
+String name
+Platform platform
+PEGI pegi
+double price
+int stock
+Date releaseDate
}

class Order {
    +int orderCode
    +double price
    +int quantity
}

class Review {
    +int reviewCode
    +double rating
    +String comment
}
```

Ademas, existe la clase `CartItem`, que representa elementos del carrito en memoria y no se persiste como entidad JPA.

## 2.4. Arquitectura de software de la solucion

La solucion sigue una estructura cercana a MVC para JavaFX, complementada con DAO y ORM.

### 2.4.1. Capa de presentacion

Compuesta por vistas FXML y controladores JavaFX.

Vistas principales:

- `LoginWindow.fxml`
- `SignUpWindow.fxml`
- `MenuWindow.fxml`

- `StoreWindow.fxml`
- `StoreAdminWindow.fxml`
- `CartWindow.fxml`
- `ReviewWindow.fxml`
- `ModifyWindow.fxml`
- `DeleteAccount.fxml`
- `DeleteAccountAdmin.fxml`
- `AddGamesWindow.fxml`
- `ModifyGameWindow.fxml`
- `HelpWindow.fxml`

### 2.4.2. Capa de logica de aplicacion

La clase `controller.Controller` actua como fachada para las operaciones funcionales del sistema, delegando en el DAO de persistencia.

Responsabilidades principales:

- Login y registro.
- Gestion de usuarios.
- Gestion del catalogo.
- Creacion de pedidos.
- Creacion de resenas.

Archivo clave: `PRG/RetoFinal/src/controller/Controller.java`

### 2.4.3. Capa de persistencia

Se implementa mediante:

- Interfaz DAO: `ClassDAO.java`
- Implementacion concreta: `DBImplementation.java`
- Gestion centralizada de sesiones: `HibernateSession.java`
- Configuracion ORM: `hibernate.cfg.xml`

Caracteristicas:

- Uso de Hibernate ORM.
- Base de datos MySQL 8.
- Estrategia `hbm2ddl.auto=update`.
- Mapeo por anotaciones JPA.
- Herencia `JOINED` para perfiles.

#### 2.4.4. Base de datos

El proyecto trabaja con la base de datos `videogame_store`.

Recursos asociados:

- Configuración de conexión: `PRG/RetoFinal/src/hibernate.cfg.xml`
- Script de datos de ejemplo: `ADT/videogame_store.sql`

Tablas conceptuales principales:

- `PROFILE_`
- `USER_`
- `ADMIN_`
- `VIDEOGAME_`
- `ORDER_`
- `REVIEW_`

#### 2.4.5. Librerías y dependencias

Dependencias observadas en el proyecto:

- JavaFX para interfaz gráfica.
- Hibernate y JPA para persistencia.
- MySQL Connector/J para conexión a la base de datos.
- Apache Commons DBCP/Pool para soporte de conexión.
- TestFX y JUnit 4 para pruebas.
- JasperReports, incluido como dependencia auxiliar.

Referencia: `PRG/RetoFinal/nbproject/project.properties`

#### 2.4.6. Flujo general de ejecución

flowchart TD

```
A[Main.java] --> B[Inicializacion Hibernate]
B --> C[Creacion o validacion de tablas]
C --> D[Creacion o validacion del admin por defecto]
D --> E[Carga de LogInWindow.fxml]
E --> F[Controladores JavaFX]
F --> G[Controller.java]
G --> H[ClassDAO / DBImplementation]
H --> I[HibernateSession]
I --> J[(MySQL videogame_store)]
```

### 3. Inventario tecnico resumido

---

#### Paquetes

- **main** : arranque de la aplicacion.
- **controller** : controladores de interfaz y fachada funcional.
- **model** : entidades, enums y persistencia.
- **view** : pantallas FXML.
- **exception** : excepcion personalizada.
- **threads** : utilidades de hilo no esenciales en el estado actual.

#### Pruebas

El proyecto incluye pruebas bajo **PRG/RetoFinal/test/** , centradas sobre todo en flujos de interfaz y controladores usando TestFX y JUnit.

### 4. Observaciones tecnicas

---

- La aplicacion crea o valida las tablas automaticamente al inicio.
- Se genera o valida un administrador por defecto con usuario **admin** y contrasena **admin** .
- El carrito de compra no es una entidad persistente, sino un estado temporal de sesion.

- El proyecto incluye manuales PDF y una plantilla JasperReports, aunque no se observa generacion de informes integrada en tiempo de ejecucion.
- Las credenciales de conexion y las contraseñas de usuario aparecen sin cifrado, aspecto mejorable desde el punto de vista de seguridad.

## 5. Conclusion

---

**Reto2CRUD\_DIN\_ADT** implementa una solucion de escritorio para la gestion funcional de una tienda de videojuegos, separando interfaz, logica y persistencia. La estructura actual permite justificar tecnicamente el proyecto desde el punto de vista del analisis funcional, el modelo de datos y la arquitectura software, cumpliendo con la base necesaria para un anexo de documentacion tecnica del sistema.